

Projekt 13. Sortowanie tablic

Autor: Patryk Stec

https://github.com/patryks-arch/Sortowanie_Alorytmy_C

1. Opis projektu i wymagań

Celem projektu było zaimplementowanie i zbadanie efektywności co najmniej czterech algorytmów sortowania, które nie opierają się na porównywaniu elementów. Program umożliwia:

- Generowanie tablic wypełnionych liczbami pseudolosowymi.
- Automatyczny pomiar czasu wykonania dla różnych wielkości ciągów ($10 < n < 1\,000\,000$).
- Prezentację wyników w konsoli.

2. Zaimplementowane algorytmy

W programie wykorzystano następujące algorytmy o złożoności zbliżonej do liniowej $O(n)$:

1. **Counting Sort (Sortowanie przez zliczanie):** Wykorzystuje tablicę pomocniczą do zliczania wystąpień każdej wartości w zbiorze.
2. **Radix Sort (Sortowanie pozycyjne):** Sortuje liczby cyfra po cyfrze, od najmniej znaczącej do najbardziej znaczącej, wykorzystując stabilny algorytm sortujący jako podprocedurę.
3. **Pigeonhole Sort:** Metoda przydatna, gdy liczba elementów i zakres możliwych wartości kluczy są zbliżone.
4. **Bucket Sort (Sortowanie kubelkowe):** Rozdziela elementy do odpowiednich "kubelków", a następnie scala je w jedną posortowaną listę.

3. Wnioski z przeprowadzonych testów:

- Wszystkie algorytmy wykonały zadanie dla miliona elementów w czasie poniżej 0.3 sekundy, co potwierdza ich wysoką wydajność w porównaniu do algorytmów klasycznych (np. bąbelkowego).
- **Najszybszym algorytmem** dla największej próby ($N=1\,000\,000$) okazał się **Counting Sort** (czas: 0.0080 s).
- Algorytm **Radix Sort** zanotował najdłuższy czas przy $N=1\,000\,000$ (0.2690 s), co wynika z konieczności wielokrotnego przepisywania tablicy przy każdej pozycji cyfry.

- Algorytmy bez porównywania doskonale sprawdzają się przy dużych zbiorach danych liczbowych o ograniczonym zakresie.

```
Test dla N = 100:  
- Counting Sort:    0.0010 s  
- Radix Sort:       0.0000 s  
- Pigeonhole Sort: 0.0000 s  
- Bucket Sort:      0.0000 s
```

```
Test dla N = 1000:  
- Counting Sort:    0.0000 s  
- Radix Sort:       0.0000 s  
- Pigeonhole Sort: 0.0000 s  
- Bucket Sort:      0.0000 s
```

```
Test dla N = 10000:  
- Counting Sort:    0.0000 s  
- Radix Sort:       0.0020 s  
- Pigeonhole Sort: 0.0010 s  
- Bucket Sort:      0.0000 s
```

```
Test dla N = 100000:  
- Counting Sort:    0.0020 s  
- Radix Sort:       0.0250 s  
- Pigeonhole Sort: 0.0020 s  
- Bucket Sort:      0.0020 s
```

```
Test dla N = 1000000:  
- Counting Sort:    0.0080 s  
- Radix Sort:       0.2690 s  
- Pigeonhole Sort: 0.0110 s  
- Bucket Sort:      0.0090 s
```